THE STUDY OF TEXT COMPRESSION ALGORITHMS AND THEIR EFFICIENCIES UNDER DIFFERENT TYPES OF FILES

Nithya P
Department of Computer Science
Pondicherry University
Puducherry, India
nithyapoupathya@gmail.com

M Sathya
Department of Computer Science
Pondicherry University
Puducherry, India
satsubithra@gmail.com

T Vengattaraman
Department of Computer Science
Pondicherry University
Puducherry, India
vengattaraman.t@gmail.com

2023 1st International Conference on Optimization Techniques for Learning (ICOTL) | 979-8-3503-2804-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICOTL59758.2023.10435164

***Abstract*— As data volumes in the digital sphere increase exponentially, it has become imperative to develop efficient means of transmitting and storing this unprecedented volume of information. Data compression techniques offer an efficient solution to reduce data sizes under limited resources; their implementation has opened the way to various domains that address issues related to storage capacity and communications bandwidth. Comprehending all available compression algorithms can be daunting due to differing requirements. Therefore, it's essential that performance evaluation of different algorithms takes place, with consideration given to compression ratio, compressed time and saved space as key performance indicators. While compression involves turning data into compact representations for storage purposes, decompression reverses this process and restores original formats of the information. As it is vitally important that no information or data are lost during compression and decompression processes, Huffman Coding (Run-length Encoding), H+R, Lempel-Ziv-Welch (Arithmetic Coding), Burrows-Wheeler Transform and Deflate are all employed here to compress text files without loss. In this paper we compare their respective algorithms such as Huffman Coding (Run-length Encoding), Lempel-Ziv-Welch (Arithmetic Coding), Burrows-Wheeler Transform and Deflate to discover their effectiveness; evaluation will include factors such as compressed file size as well as compression ratio in addition to time and space savings considerations.**

Keywords*— *Data compression, Lossless, Huffman coding, Run-length encoding, Lempel-Ziv-Welch, Arithmetic Coding, burrows-wheeler transformer, Deflate.

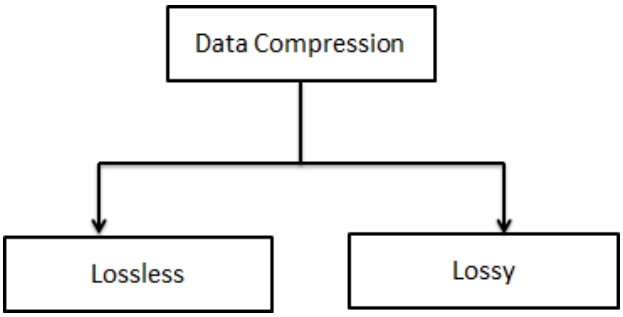
I. Introduction

Data compression is the process of transforming an input data stream that may be large into one that is smaller, using less bits to represent data and saving storage costs by making more information fit within available storage space. Furthermore, compression can improve transmission speed leading to more efficient transfers and transfers. Imagine that your phone memory is full. While increasing storage capacity might not always be feasible, you can free up space by deleting older images through data compression.

A text consists of many words, and words themselves comprise numerous characters, so all of the characters must be stored in order to store the text. It takes a lot of space to store this kind of stuff. Therefore, a method to minimize the amount of data such that it takes up less space is required. The main goal of text compression is to reduce the size of text files while preserving the original content and format. It can be used in various applications, such as email, messaging, and online content delivery, where reducing the size of text files can help to improve transmission times and reduce storage costs. The repetition of words and phrases in children's rhymes and stories makes them excellent instances of text compression [1].

Text compression algorithms work by transforming strings of characters into one that has been compressed while still maintaining all its information, yet reduced in size by an appreciable amount. Metrics such as compression size or ratio and processing speed are used to measure this.

In this paper, we compare the different algorithms for different types of text files and compare the compressed file size and other metrics. Data compression is divided into two categories: a) Lossless Compression b) Lossy Compression



A. Lossless compression

Lossless compression allows the reconstruction of original data with no loss in information, or uncompressing a compressed file using lossless compression restores all original information. Lossless compression works best in situations when original information is essential such as text and data files.

Researchers must conduct extensive studies of text lossless compression to improve efficiency of lossless algorithms and advance compression technologies, creating more efficient algorithms by studying its intricacies. By understanding lossless text compression mechanisms like ZIP, PNG and GIF compression algorithms which have become widely utilized data compression solutions; researchers may create more efficient lossless text algorithms and ensure data remains faithful when decompressed [2].

B. Lossy compression

Lossy compression reduces file sizes by permanently erasing certain portions of their original data, leaving behind only fragmented copies when uncompressed; this technique allows higher compression rates at reduced quality - its impact depending on which compression algorithm and desired rate is chosen.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE
Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

Lossy compression algorithms are most frequently seen used for media files such as audio and video because their quality loss can easily be tolerated by humans while still producing smaller files - JPEG and MP3 are among many such examples of lossy algorithms [2].

II. LITERATURE SURVEY

Zuchao Li et al. have done much research on this subject. [3] is dedicated to improving language representation by employing text-based foundational knowledge, with explicit and implicit text compressing methods being proposed as ways of creating more precise statistical distributions; additionally, these techniques are integrated in Transformer encoders in order to increase quality representations generated within language domains.

Manak Pandey et al. [4] delve into a novel hybrid methodology that integrates Run Length Encoding (RLE), Infix Encoding, and Bit Reduction techniques. The primary aim of this approach is to achieve the highest possible compression ratio for text files while also encompassing dictionary information.

Abu Hila et al. [5] present an approach for compressing 2-byte Turkish characters into one byte using lossless text compression, by employing sequential encoding methods to convert from UTF-8 coding to ANSI character encodings.

Harwin C. Mendoza and colleagues developed an Improved Hamming Code Data Compression Method in order to mitigate inflation by decreasing the number of bits required to replace an invalid codesword with valid ones, and to maximize bit usage during compression. This technique makes the most out of every compression effort available.

Md. Ashiq Mahmood et al. [7] present an efficient method for compressing large text using 5 Bit Encoding Scheme (5BE), which converts characters from 8 bits to 5 bits. They employ Dictionary Method in addition to 5BE Compression System to achieve compressibility of text.

Maaheer Amaeer et al. [8] focused their research on compressing 16-bit Unicode Bangla text including English characters using an 8 bit encoding, using binary values sequences to form an efficient bitstream.

Tariq Abu Hilal and colleagues' [9] work aims at compressing Arabic characters from 2 bytes into 1 using lossless text compression, using sequential encoding as a means to convert UTF-8 character coding to ANSI character encoding and then compress.

Radu Radescu and colleagues evaluated the performance of Q-Coder to compare different compression efficiency levels for Romanian and English text files [10] using Q-coder.

Md Farhad Mokter and colleagues [11] employ an effective technique for representing and compressing single and conjugate Bengali characters. For single characters they utilize a two-digit decimal number while four-digit decimals numbers are employed when compressing conjugate characters. Character indexing methods are then employed in compression/decompression operations.

B. Vijayalakshmi et al. present an innovative method of lossless text compression of Tamil texts using Unicode characters, taking into account both transmission and storage constraints. They utilize an optimal set for transmitting and storing memory storage requirements.

Ashiq Mahmood et al. introduce the 6-bit Encoding (6BE) technique for text data compression. This technique encodes each character using six bits for easier reading. They propose an algorithm which provides lossless compression of 6-BE strings by Huffman or LZW compression methods.

Shaurya Agarwal and colleagues [14] explore wavelet transforms-based methods for compressing Intelligent Transportation System data using Bangla Text Compression with a modified Lempel Ziv Welch algorithm.

Linkon Barua et al. [15] have utilized an alternative algorithm called MLZW for efficient compression of Bangla texts. This technique encodes each character according to its dictionary index value.

Ibrahim S Alkhazi et al. [16] demonstrate the effectiveness of Prediction-by-Partial Matching (PPM) compression for categorizing and segmenting Arabic text, using parts-of-speech tags combined with text to compress. Before transmission, compressed text is tagged with words and tags to ensure maximum compression efficiency using PPM text-compression scheme.

Sourya Basu and colleagues [17] employed character frequency-based encoding to achieve maximum retention through text compression algorithm, thus decreasing entropy.

Mohammed Javed et al. [18] propose an innovative technique for segmenting printed text documents simultaneously into words and characters using run-length compress. They employ substitution to extract intelligently column runs from each row. Huffman coding is employed during compression/decompression operations as well as run-length compressing to achieve these goals.

Abu Marjan, Md. and their colleagues use an effective, lossless technique for representing and compressing large Bengali texts. Character indexing is used both during compression and decompression processes.

Melbin Kuruvila and coworkers use Huffman coding to compress Malayalam texts, employing a method which incorporates both Malayalam alphabets and English letters, in addition to arithmetic numbers; their code selects only likely characters by representing them with less bits.

Nguyen Quoc Viet Hung and colleagues [21] conducted an in-depth empirical comparison between various model-

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

based compression methods, and evaluated their performance using cutting edge methods.

Abhishek Kumar and colleagues [22] attempt to reduce data size by developing an approach using English as their reference language and applying Static Dictionary methods.

Shawki A. et al. [23] investigate the impact of various factors on text compressibility, including family of wavelet filter, decomposition level, compression size and wavelet threshold. Wavelet and Fourier transformations prove particularly effective at compressing non-stationary text data for compression purposes.

TABLE 1- COMPARISON OF DIFFERENT METRICS

Paper No	Compression Ratio	Compression Factor	Compression Time	Compression Speed	Compression Percentage	Space Savings	Error Rate
[3]	✓	✓	×	×	×	×	×
[4]	✓	×	×	×	×	✓	×
[5]	×	×	×	×	✓	×	×
[6]	✓	×	×	✓	×	×	×
[7]	✓	×	✓	×	×	×	✓
[8]	✓	×	×	×	×	×	×
[9]	×	×	×	×	✓	×	×
[10]	×	×	×	×	×	×	×
[11]	✓	×	×	×	×	×	×
[12]	×	×	×	×	✓	×	×
[13]	✓	×	×	×	×	✓	×
[14]	×	×	✓	✓	×	×	×
[15]	✓	×	×	×	×	×	✓
[16]	×	✓	×	×	×	×	×
[17]	×	×	×	×	×	×	×
[18]	✓	×	×	×	×	×	×
[19]	×	✓	×	×	×	×	×
[20]	✓	×	×	✓	×	✓	×
[21]	✓	×	×	×	×	×	×
[22]	✓	×	✓	×	×	×	×
[23]	✓	×	×	×	×	×	×

III. COMPRESSION ALGORITHMS

A. Huffman Coding

A popular lossless text data compression technology is huffman coding. It was created in 1952 by David A. Huffman and is based on the idea that less often appearing characters should be given longer codes and more frequently occurring characters should be given shorter codes.

The input text is first examined by the method in Huffman coding, which then creates a binary tree that depicts the frequency distribution of characters in the text. The way the tree is built makes sure that the shortest binary codes are given to the characters that occur the most frequently.

Each character in the input text is given a distinct binary code based on its location in the binary tree once it has been created. The original character is then changed in the compressed text using this code. In real time example of Huffman algorithm is used in data transmission using fax and text, it reduces the size and improves the speed of transmission.

Algorithm 1 HUFFMAN CODING [24]

1: Create a priority queue Q consisting of each unique character.
2: Sort then in ascending order of their frequencies.
3: For all the unique characters:
   a. create a newNode
   b. for character in input_string:
   c. extract minimum value from Q and assign it to leftChild of newNode
   d. extract minimum value from Q and assign it to rightChil of newNode
   e. Calculate the sum of these two minimum values and assign it to the value of new Node
   f. insert this new Node into the tree
4: return root Node

B. Lempel-Ziv Welch

A lossless data compression algorithm called Lempel-Ziv-Welch (LZW) functions by substituting a dictionary of shorter code words for recurrent patterns in the input data. After reading the input data, the algorithm fills a dictionary with every conceivable symbol. The next step is to find the longest string of symbols that is already present in the dictionary by reading each symbol in the incoming data one at a time. A sequence is substituted with the relevant code word from the dictionary when it is discovered. The sequence is substituted with a new code word that is added to the dictionary if the sequence is not already there.

LZW is a popular text compression method that has been used in a variety of file types, including some PDF files, the GIF image format, and the UNIX compress command. The algorithm has been demonstrated to obtain high compression ratios for specific types of input data and is effective at compressing text material that contains repeated patterns. On other data types, though, including random data or data with little redundancy, it might not work well. Compression utilities like gzip, GIF image compression, and the V.42 modem standard extensively employ Lempel-Ziv algorithms.

C. Arithmetic Coding

Arithmetic coding is a lossless data compression algorithm that aims to achieve high compression ratios by encoding a message as a single fractional number between 0 and 1.The basic goal of arithmetic coding is to give each symbol in the input message a range based on the likelihood that it will appear. A lower bound and an upper bound within the range [0, 1] define the range. At first, the input message is given the complete range [0, 1].

To encode a symbol, the algorithm updates the range based on the probability distribution of the symbols in the message. The more probable a symbol, the larger its assigned range within the current interval. This process is repeated for each symbol in the input message, narrowing down the interval with each step.

At the decoding stage, the fractional number representing the encoded message is converted back into the

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

original sequence of symbols. The decoder uses the same probability distribution to reconstruct the intervals and determine the symbols.

Algorithm 3 ARITHMETIC CODING [26]

```
/*ARITHMETIC ENCODING ALGORITHM. */
1: Encoded_symbol(symbol, c_freq)
      range = high - low
      high = low + range*c_freq[symbol-1]
      low = low + range*c_freq[symbol]
/*ARITHMETIC DECODING ALGORITHM. */
2: decode_symbol(sum_freq)
      find symbol such that
            cum_freq[symbol] <= (value - low) /
            (high - low) < c_freq[symbol -1]
       range = high - low
       high = low + range*c_freq[symbol-1]
       low = low + range*c_freq[symbol]
3: return symbol
```

D. Runlength encoding

A lossless text data compression algorithm called run-length encoding finds repeated character sequences and encodes them with a single symbol that represents the sequence. Run-length encoding involves scanning the input text to find groups of identical characters. The algorithm records the character's appearances in each sequence in order of frequency, then the character itself. The algorithm, for instance, would encode the sequence "AAAAA" in the input text as "5A".

Run-length coding can be an efficient method for compressing text data such as binary or image data containing long sequences of repeated characters or patterns, providing high compression rates with relatively simple encoding techniques. Run-length compression may not work for all data types as some input files contain patterns unsuitable for run-length compression; however, it can still provide efficient storage and transmittal of this information.

Algorithm 4 RUNLENGTH ENCODING [27]

```
1: procedure Run-length Encoding
2: input_string ← string to be encoded
3: output_string ← ""
4: current_char ← first character in input_string
5: current_count ← 1
6: for each character c in input_string starting from second
character do
7: if c is the same as current_char then
8: current_count ← current_count + 1
9: else
10: output_string ← output_string + current_char + current_count
11: current_char ← c
12: current_count ← 1
13: end for
14: output_string ← output_string + current_char + current_count
15: return output_string
16: end procedure
```

E. H+R

H+R algorithm is a hybrid. It compresses files using Run-Length Encoding and Huffman Coding techniques to reduce file sizes. After reading an input text file, RLE is applied in order to reduce repetitive characters before counting its frequency counted by frequency analysis. Based on character frequency, a Huffman tree can be developed. From there, Huffman codes for each character can be generated through this tree and used to encode RLE text into compressed format. This algorithm calculates compression time, original file size and compressed file sizes; converts file sizes to appropriate units; provides information about compression ratio and space saving; applies Huffman codes and RLE decoding to decompress files quickly; decompressed text may then be saved into an output file for subsequent use by this RLE and Huffman-code based algorithm for efficient file compression/decompression;

Algorithm 5 H+R

1: Implement RLE functions for compression and decompression.

2: Determine how many characters exist within the input text.

3: Construct a Huffman Tree using character frequencies.

4: Huffman codes derived from Huffman trees for each character

5: Use Huffman codes to encode text.

6: Calculate Metrics Ask user for path of input file

7: Text from an input file will appear as expected.

8: Compressing text using RLE and Huffman codes.

9: Unlock compressed text using Huffman Tree technology.

F. Burrows–Wheeler transform

Michael Burrows & David Wheeler first created the Burrows Wheeler Transform (BWT), a text transformation technique, in 1994. Rearranging characters within a string to improve compression and enable effective pattern matching algorithms is the goal of BWT; to accomplish this it rotates characters cyclically while sorting lexicographically before selectinging each final character of every rotation and placing alphabetically; this transformative technique may result in grouping repeated characters or even long runs of identical ones.

Algorithm 6 BURROWS-WHEELER TRANSFORM [28]

```
1: function BWT (string s)
      Make a table showing each possible rotation of an s in its
various states of motion.
      sort rows alphabetical order
      return (end column of the table)
2: function inverseBWT (string s)
```

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

create an empty table
repeat length(s) times
// first insert creates first column
Add the suffix s before the column name in a table.

Sort each row alphabetically for alphabetic ordering.
3: return (row that ends with the 'En Of File' character)

G. Deflate algorithm

DEFLATE compression algorithm is one of the most frequently utilized compression techniques. It combines LZ77 string matching and Huffman as an entropy encoder. Popular file formats, including ZIP, Gzip and PNG files utilize this compression method for optimal compression results.

Algorithm 7 DEFLATE [11]

1: Initialize a sliding window and a look ahead buffer.
2: While there are input symbols remaining:
   a. Utilize the LZ77 algorithm to identify the most extended matching substring between the sliding window and the look-ahead buffer.
   b. Output a token representing the match or a literal symbol.
   c. Move the sliding window and lookahead buffer accordingly.
3: After processing all input symbols, add an end-of-block token.
4: Assign the prefix 0 to the subset with higher probability and 1 to the subset with lower probability
5: Add additional compression features such as block splitting, dynamic Huffman tables, and run-length encoding.
6: Generate the compressed output stream.

IV. EXPERIMENTS AND RESULT ANALYSIS

A. DATASETS

Traditionally employed for testing data compression programs, the Calgary Corpus comprises a set of text and binary data files. The Calgary Corpus consists of a set of files that cover a wide range of topics, including news articles, technical documents, fiction, etc. And contains text, image, and object files. It is often used as a benchmark dataset for evaluating the performance of compression algorithms, as it provides a diverse set of text data with varying characteristics. Here some of the Standard text datasets are taken from the Calgary Corpus.

TABLE 2- CALGARY CORPUS

Paper ID	File name	Size (bytes)	Description
[3]	bib	111,261	ASCII text in UNIX "refer" format-725 bibliographic references
[4]	news	377,109	ASCII text–USENET batch file on a variety of topics.
[5]	Paper 1	53,161	UNIX "troff" format–Written, Neal, Cleary: Arithmetic Coding for Data Compression.
[6]	Paper 2	82,199	UNIX "troff" format–Written: Computer (in) security.
[7]	Book 1	768,771	The unformatted ASCII text corresponds to the novel "Far from the Madding Crowd" written by Thomas Hardy.
[8]	Book 2	610,856	ASCII text in UNIX "troff" format-Witten: Principles of Computer Speech
[9]	Trans	93,695	The transcript of a terminal session consists of ASCII characters, including control characters.

B. PERFORMANCE METRICS

1. Compression Ratio

Compression ratio is a measure of how much a file has been compressed in comparison to its original size. Higher compression ratio is considered better because it indicates more efficient use of storage space.

Compression ratio = Compressed File Size/Original File Size.

2. Compression Time

The compression time refers to the duration required for compressing a file using a compression algorithm. The best compression time would be the shortest possible time needed to achieve the desired compression ratio.

Compression time = Size of the input file/Compression speed.

3. Space Saving

Space savings represents the percentage of space saved compared to the original file size. It quantifies the efficiency of the compression algorithm in utilizing storage space.

*Space Savings = (1 - Compressed File Size / Original File Size) * 100*

C. EXPERIMENTATIONS

Implementations have done for seven different lossless algorithms such as Huffman Coding, Lempel-Ziv Welch, The techniques employed for text compression include Arithmetic Coding, Run-length Encoding, and the Burrows- Wheeler Transform, Deflate and a hybrid algorithm of H+R with 7 different types of files that are taken from the Calgary Corpus. For each algorithm, these seven different files are implemented and compared. The output will display the Original File Size, Compression Ratio , Compressed File Size, Compression Time and Space Savings.

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

Table 3 shows a Comparison table in terms of File Size. All the original and compressed size of the entire file is given. Table 4 shows a Comparison table in terms of Metrics. The performance metrics of Compression ratio, Compression Time and Space Savings is given.

TABLE 3- COMPARISON OF DATASETS WITH DIFFERENT ALGORITHMS IN TERMS OF FILE SIZE

Paper id	File Size	Huffman	LZW	Arithmetic
[3]	Input Size:	109 KB	109 KB	109 KB
	Compressed Size:	71 KB	52 KB	14 KB
[4]	Input Size:	368 KB	368 KB	368 KB
	Compressed Size:	241 KB	178 KB	46 KB
[5]	Input Size:	52 KB	52 KB	52 KB
	Compressed Size:	33 KB	30 KB	6KB
[6]	Input Size:	80 KB	80 KB	80 KB
	Compressed Size:	46KB	42 KB	10 KB
[7]	Input Size:	751 KB	751 KB	751 KB
	Compressed Size:	428 KB	301 KB	94 KB
[8]	Input Size:	597 KB	597 KB	597 KB
	Compressed Size:	360 KB	236 KB	75 KB
[9]	Input Size:	91 KB	91 KB	91 KB
	Compressed Size:	62 KB	43 KB	11 KB

Paper id	File Size	Run-length	H+R	BWT	Deflate
[3]	Input Size:	109 KB	109 KB	109 KB	109 KB
	Compressed Size:	224 KB	96 KB	109 KB	34 KB
[4]	Input Size:	368 KB	368 KB	368 KB	368 KB
	Compressed Size:	710 KB	318 KB	368 KB	141 KB
[5]	Input Size:	52 KB	52 KB	52 KB	52 KB
	Compressed Size:	105 KB	45 KB	52 KB	18 KB
[6]	Input Size:	80 KB	80 KB	80 KB	80 KB
	Compressed Size:	160 KB	66 KB	80 KB	29 KB
[7]	Input Size:	751 KB	751 KB	751 KB	751 KB
	Compressed Size:	1MB	616 KB	751 KB	306 KB
[8]	Input Size:	597 KB	597 KB	597 KB	597 KB
	Compressed Size:	1 MB	505 KB	597 KB	202 KB
[9]	Input Size:	91 KB	91 KB	91 KB	91 KB
	Compressed Size:	176 KB	78 KB	90 KB	19 KB

TABLE 4- COMPARISON OF DATASETS WITH DIFFERENT ALGORITHMS IN TERMS OF METRICS

Paper id	File Size	Huffman	LZW	Arithmetic
[3]	Compression Ratio:	0.65	0.48	0.12
	Compression Time:	0.08 sec	0.06 sec	0.19 sec
	Space Savings:	35 %	52 %	88 %
[4]	Compression Ratio:	0.65	0.48	0.12
	Compression Time:	0.29 Sec	0.13 sec	1.52sec
	Space Savings:	35 %	52 %	88 %
[5]	Compression Ratio:	0.63	0.58	0.12
	Compression Time:	0.05 sec	0.03 sec	0.05 sec
	Space Savings:	37 %	42 %	88 %
[6]	Compression Ratio:	0.58	0.52	0.12
	Compression Time:	0.05 sec	0.04 sec	0.12 sec
	Space Savings:	42 %	48 %	58 %
[7]	Compression Ratio:	0.57	0.40	0.12
	Compression Time:	0.49 sec	0.25 sec	5.84 sec
	Space Savings:	43 %	60 %	88 %
[8]	Compression Ratio:	0.60	0.40	0.12
	Compression Time:	0.41 sec	0.20 sec	3.75 sec
	Space Savings:	40 %	60 %	88 %
[9]	Compression Ratio:	0.68	0.47	0.12
	Compression Time:	0.08 sec	0.04 sec	0.14 sec
	Space Savings:	32 %	53 %	88 %

Paper id	File Size	Run-length	H+R	BWT	Deflate
[3]	Compression Ratio:	2.06	1.12	1.00	0.32
	Compression Time:	0.06 sec	0.07 sec	0.34 sec	0.0 sec
	Space Savings:	-106 %	11.27%	0 %	68 %
[4]	Compression Ratio:	1.92	1.15	1.00	0.38
	Compression Time:	0.17 sec	0.93 sec	1.22 sec	0.02 sec
	Space Savings:	-93%	13.51%	0 %	62 %
[5]	Compression Ratio:	2.01	1.15	1.00	0.35
	Compression Time:	0.03 sec	0.03 sec	0.17 sec	0.0 sec
	Space Savings:	-102 %	13.52 %	0 %	65 %
[6]	Compression Ratio:	1.98	1.21	1.00	0.36
	Compression Time:	0.03 sec	0.04 sec	0.25 sec	0.0 sec
	Space Savings:	-99 %	17.23 %	0 %	64 %
[7]	Compression Ratio:	1.97	1.22	1.00	0.41
	Compression Time:	0.42 sec	3.64 sec	2.53 sec	0.05 sec
	Space Savings:	-98 %	18 %	0 %	59 %
[8]	Compression Ratio:	2.00	1.18	1.00	0.34
	Compression Time:	0.30 sec	2.42 sec	1.97 sec	0.03 sec
	Space Savings:	-100 %	15 %	0 %	66 %
[9]	Compression Ratio:	1.92	1.18	0.98	0.20
	Compression Time:	0.03 sec	0.04 sec	0.92sec	0.0 sec
	Space Savings:	-92%	16 %	2.14 %	80 %

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

D. RESULT ANALYSIS

The Run-length encoding algorithm and Burrows-Wheeler Transform algorithm were not compressed the files. Run-length provides a higher compression size than the original file size and the Burrows-Wheeler Transform gives the same compressed size as of the input file is shown in Table 3.

Because, Run-length algorithm is a simple compression technique that works well for certain types of data, particularly when there are long runs of repeated characters. However, it may not be efficient for all types of data. If the input file contains patterns that are not well-suited for run-length compression. It works well in Image data and also the efficiency of the BWT algorithm depends on the characteristics of the input data. If the data has little redundancy or does not exhibit favorable patterns for compression, the BWT alone may not yield significant compression gains or may even increase the file size.

The Run-length algorithm alone not compressed the dataset well. So, the combination of Huffman and Run- length (H+R) hybrid algorithm is used and compared is shown in Table 3 and Table 4. This hybrid algorithm and the remaining four algorithms are compared with their metrics and shown in the graph.

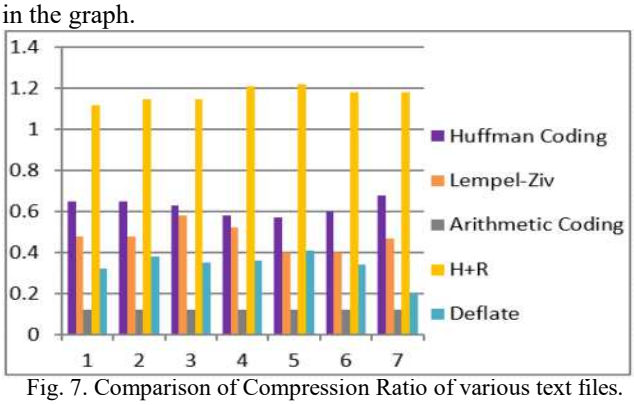


Fig. 7. Comparison of Compression Ratio of various text files.

From the Figure 7, it is seen that the **Arithmetic Coding** provides an efficient Compression Ratio in all files.

After that Deflate algorithm provides lesser compression ratio in compared to other algorithms and then, Lempel-Ziv algorithm and Huffman algorithm provides an average compression Ratio. But, in File 5(Book 1) both the Deflate algorithm and Lempel-Ziv algorithm provides same level of compression ratio.it is seen that Lempel-Ziv provides lesser compression ratio for large files. H+R algorithm gives a higher Compression Ratio in compared to all the files.

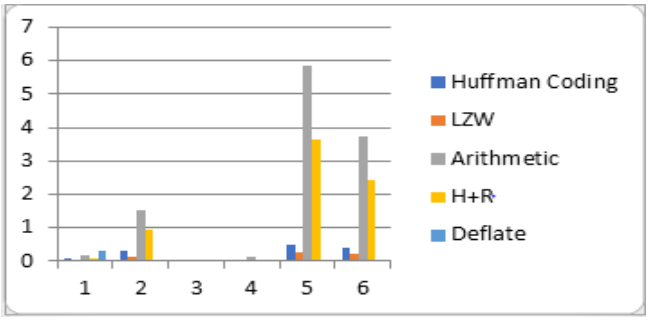


Fig. 8. Comparison of Compression Time of various text files.

From the Figure 8, it is seen that the **Deflate algorithm** takes a lesser Compression Time in all the files. It provides an efficient Compression Time.

After that, Lempel-Ziv Welch (LZW) algorithm takes a lesser Compression Time in compared to all other algorithms and then Huffman algorithm provides a lesser compression time. Arithmetic and H+R algorithm takes a higher Compression Time to all the files.

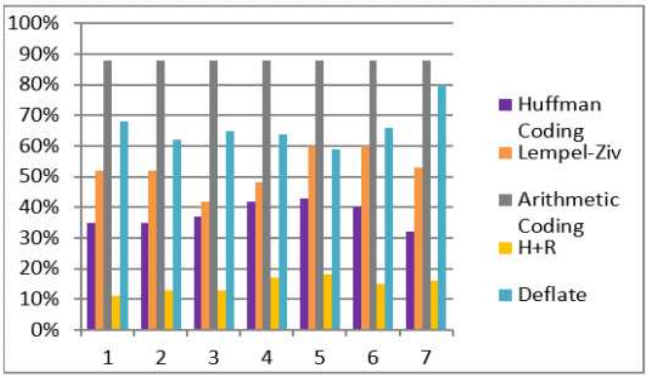


Fig. 9. Comparison of Space Saving of various text files.

From the Figure 9, it is seen that the **Arithmetic Coding** provides a higher Space saving in all the files. It provides an efficient Space saving.

After that Deflate algorithm provides a higher Space saving in compared to other algorithms and then Lempel- Ziv and Huffman algorithm takes an average Space Saving. H+R algorithm provides a lesser Space saving in compared to all files.

.

V. CONCLUSION

This study explores lossless text compression approaches and evaluates the effectiveness of different algorithms on various text files. It also evaluates the performance of several compression algorithms, namely Huffman coding, Lempel-Ziv Welch, Arithmetic coding, Run-Length Encoding, Hybrid algorithm (H+R), Burrows-Wheeler transform and Deflate algorithm. The aim of the study is to determine the most efficient algorithm for compressing data in the taken dataset. By analyzing the compression results obtained from each algorithm, the study makes conclusions regarding their efficiency. The study finds that the Arithmetic algorithm provides a higher efficiency in compressing the dataset but, takes higher compression time.

References

[1] S. Shanmugasundaram and R. Lourdusamy, "A Comparative Study Of Text Compression Algorithms," vol. 1, 2011.

[2] Tanvi Patel, Judith Angela, Poonam Choudhary, Kruti Dangarwala, and Sad Vidhya Mandal Institue of Technology, "Survey of Text Compression Algorithms," *IJERT*, vol. V4, no. 03, p. IJERTV4IS030932, Mar. 2015, doi: 10.17577/IJERTV4IS030932.

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.

[3] Z. Li *et al.*, "Text Compression-aided Transformer Encoding," *IEEE Trans. Pattern Anal. Mach. Intell.*, pp. 1–1, 2021, doi: 10.1109/TPAMI.2021.3058341.

[4] M. Pandey, S. Shrivastava, S. Pandey, and S. Shridevi, "An Enhanced Data Compression Algorithm," in *2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*, Vellore, India: IEEE, Feb. 2020, pp. 1–4. doi: 10.1109/ic-ETITE47903.2020.223.

[5] T. A. Hilal and H. A. Hilal, "Turkish Text Compression via Characters Encoding," *Procedia Computer Science*, vol. 175, pp. 286–291, 2020, doi: 10.1016/j.procs.2020.07.042.

[6] H. C. Mendoza, A. M. Sison, and R. P. Medina, "Modifying Bit-Level Data Compression Scheme based on Adaptive Hamming Code Data Compression Algorithm," in *2019 International Conference on Multimedia Analysis and Pattern Recognition (MAPR)*, Ho Chi Minh City, Vietnam: IEEE, May 2019, pp. 1–4. doi: 10.1109/MAPR.2019.8743541.

[7] Md. A. Mahmood and K. M. A. Hasan, "A Dictionary based Compression Scheme for Natural Language Text with Reduced Bit Encoding," in *2019 IEEE International Conference on Robotics, Automation, Artificial-intelligence and Internet-of-Things (RAAICON)*, Dhaka, Bangladesh: IEEE, Nov. 2019, pp. 145–150. doi: 10.1109/RAAICON48939.2019.62.

[8] M. Amaeer, K. M. A. Hasan, Md. Ashiq Mahmood, P. Shuvro, and A. Awal, "An Efficient Compression Scheme for Bangla Natural Text by 16 Bit Unicode Transformation," in *2019 2nd International Conference on Innovation in Engineering and Technology (ICIET)*, Dhaka, Bangladesh: IEEE, Dec. 2019, pp. 1–6. doi: 10.1109/ICIET48527.2019.9290703.

[9] T. A. Hilal and H. A. Hilal, "Arabic Text Lossless Compression by Characters Encoding," *Procedia Computer Science*, vol. 155, pp. 618–623, 2019, doi: 10.1016/j.procs.2019.08.087.

[10] R. Radescu, "Comparative Study of Performances in Lossless Data Compression for English and Romanian Text Files Using the Q-Coder," in *2018 International Symposium on Fundamentals of Electrical Engineering (ISFEE)*, Bucharest, Romania: IEEE, Nov. 2018, pp. 1–4. doi: 10.1109/ISFEE.2018.8742466.

[11] M. Farhad Mokter, S. Akter, Md. Palash Uddin, M. Ibn Afjal, Md. Al Mamun, and Md. Abu Marjan, "An Efficient Technique for Representation and Compression of Bengali Text," in *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)*, Sylhet: IEEE, Sep. 2018, pp. 1–6. doi: 10.1109/ICBSLP.2018.8554838.

[12] B. Vijayalakshmi, "Lossless Text Compression Technique Based on Static Dictionary for Unicode Tamil Document".

[13] A. Mahmood, T. Latif, and K. M. A. Hasan, "An efficient 6 bit encoding scheme for printable characters by table look up," in *2017 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, Cox's Bazar, Bangladesh: IEEE, Feb. 2017, pp. 468–472. doi: 10.1109/ECACE.2017.7912950.

[14] S. Agarwal, E. E. Regentova, P. Kachroo, and H. Verma, "Multidimensional Compression of ITS Data Using Wavelet-Based Compression Techniques," *IEEE Trans. Intell. Transport. Syst.*, vol. 18, no. 7, pp. 1907–1917, Jul. 2017, doi: 10.1109/TITS.2016.2613982.

[15] L. Barua, P. K. Dhar, L. Alam, and I. Echizen, "Bangla text compression based on modified Lempel-Ziv-Welch algorithm," in *2017 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, Cox's Bazar, Bangladesh: IEEE, Feb. 2017, pp. 855–859. doi: 10.1109/ECACE.2017.7913022.

[16] I. S. Alkhazi, M. A. Alghamdi, and W. J. Teahan, "Tag based models for Arabic text compression," in *2017 Intelligent Systems Conference (IntelliSys)*, London: IEEE, Sep. 2017, pp. 697–705. doi: 10.1109/IntelliSys.2017.8324370.

[17] S. Basu, S. Chaturvedi, and R. M. Hegde, "Text compression using lexicographic permutation of binary strings," in *2016 International Conference on Signal Processing and Communications (SPCOM)*, Bangalore, India: IEEE, Jun. 2016, pp. 1–5. doi: 10.1109/SPCOM.2016.7746651.

[18] M. Javed, P. Nagabhushan, and B. B. Chaudhuri, "Extraction of line-word-character segments directly from run-length compressed printed text-documents," in *2013 Fourth National Conference on Computer Vision, Pattern Recognition, Image Processing and Graphics (NCVPRIPG)*, Jodhpur, India: IEEE, Dec. 2013, pp. 1–4. doi: 10.1109/NCVPRIPG.2013.6776195.

[19] Md. A. Marjan, Md. P. Uddin, M. I. Afjal, and Md. D. Haque, "Developing an efficient algorithm for representation and compression of large Bengali text," in *2014 9th International Forum on Strategic Technology (IFOST)*, Cox's Bazar, Bangladesh: IEEE, Oct. 2014, pp. 22–25. doi: 10.1109/IFOST.2014.6991063.

[20] M. Kuruvila and D. P. Gopinath, "Entropy of Malayalam language and text compression using Huffman coding," in *2014 First International Conference on Computational Systems and Communications (ICCSC)*, Trivandrum, India: IEEE, Dec. 2014, pp. 150–155. doi: 10.1109/COMPSC.2014.7032638.

[21] N. Q. V. Hung, H. Jeung, and K. Aberer, "An Evaluation of Model-Based Approaches to Sensor Data Compression," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 11, pp. 2434–2447, Nov. 2013, doi: 10.1109/TKDE.2012.237.

[22] A. Kumar, S. M. Ahmed, and V. K. Duleb, "English text compression for small messages," in *ICIMU 2011 : Proceedings of the 5th international Conference on Information Technology & Multimedia*, Kuala Lumpur, Malaysia: IEEE, Nov. 2011, pp. 1–5. doi: 10.1109/ICIMU.2011.6122737.

[23] S. A. Al-Dubaee and N. Ahmad, "Multilingual Lossy Text Compression Using Wavelet Transform," in *2010 First International Conference on Integrated Intelligent Computing*, Bangalore, India: IEEE, Aug. 2010, pp. 39–44. doi: 10.1109/ICIIC.2010.65.

[24] R. Arshad, A. Saleem, and D. Khan, "Performance comparison of Huffman Coding and Double Huffman Coding," in *2016 Sixth International Conference on Innovative Computing Technology (INTECH)*, Dublin, Ireland: IEEE, Aug. 2016, pp. 361–364. doi: 10.1109/INTECH.2016.7845058.

[25] L. S. Tan, S. P. Lau, and C. E. Tan, "Optimizing LZW text compression algorithm via multithreading programming," in *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, Kuala Lumpur, Malaysia: IEEE, Dec. 2009, pp. 592–596. doi: 10.1109/MICC.2009.5431427.

[26] I. H. WIllEN, R. M. Neal, and J. G. Cleary, "ARITHMETIC CODING FOR DATA COIUPRESSION," *Communications of the ACM*, vol. 30, no. 6, 1987.

[27] K. E. Prasetyo, T. W. Purboyo, and R. E. Saputra, "Comparison of Text Data Compression Using Run Length Encoding, Arithmetic Encoding, Punctured Elias Code and Goldbach Code," *IJET*, vol. 9, no. 5, pp. 3821–3827, Oct. 2017, doi: 10.21817/ijet/2017/v9i5/170905151.

[28] A. Moffat and R. Y. K. Isal, "Word-based text compression using the Burrows–Wheeler transform," *Information Processing & Management*, vol. 41, no. 5, pp. 1175–1192, Sep. 2005, doi: 10.1016/j.ipm.2004.08.009.

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on February 29,2024 at 19:29:18 UTC from IEEE Xplore. Restrictions apply.